ИНЖЕНЕРНАЯ ПРЕЗЕНТАЦИЯ

Agentic Harness для написания кода

Как LLM превращается в управляемого coding agent

РАСХОЖДЕНИЕ ОЖИДАНИЙ

Одной LLM недостаточно для надёжных изменений в кодовой базе

Chat completion

- однократный ответ по промпту
- ограниченное «окно» контекста
- нет гарантированного доступа к файлам и среде
- нет воспроизводимого цикла проверки

Agentic coding system

- управляемый цикл: план → действие → наблюдение
- явные инструменты с политиками
- патчи, тесты, логи как часть процесса
- память состояния и аудит шагов

Модель генерирует гипотезы; без обвязки их нельзя систематически превратить в проверяемые изменения.

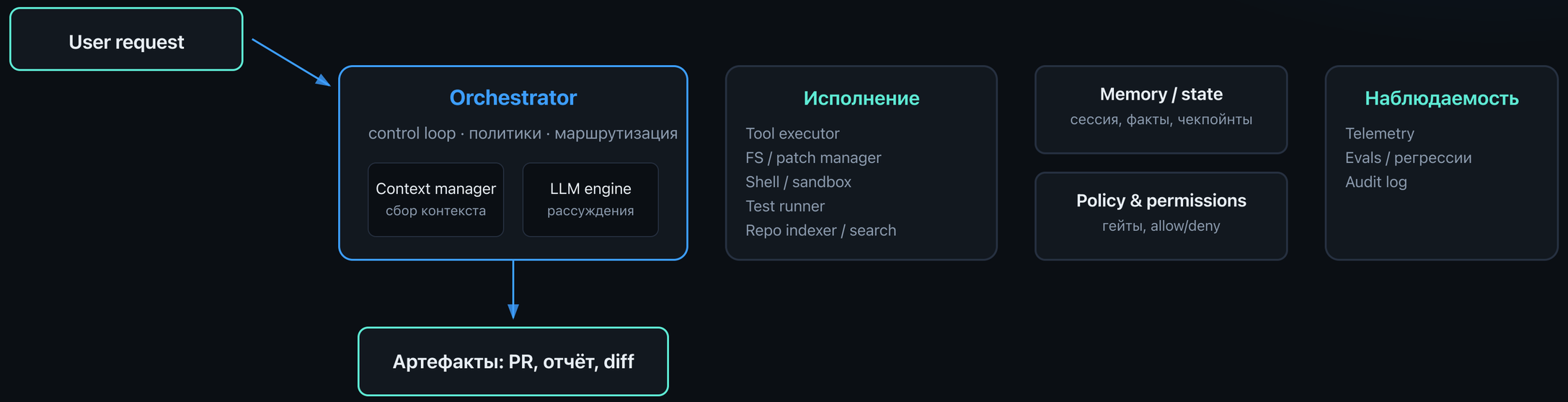
ОПРЕДЕЛЕНИЕ

Harness — исполнительно-управляющая обвязка вокруг модели, не «ещё один промпт»

- **Reasoning core:** LLM предлагает шаги, объяснения, планы и интерпретацию ошибок.
- **Execution / control layer:** оркестратор вызывает инструменты, соблюдает политику, фиксирует артефакты.
- Harness превращает модель в исполнителя задач: читает код, правит файлы, гоняет тесты, корректирует диффы.

КОМПОНЕНТЫ

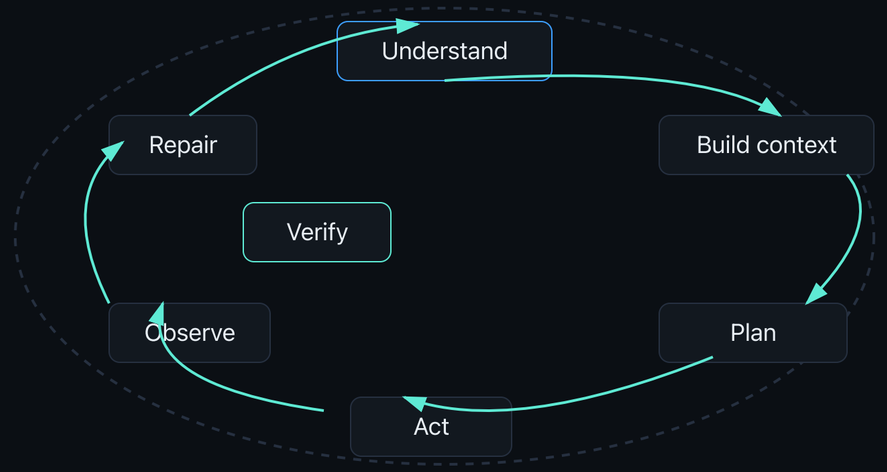
Надёжность coding agent распределена между десятком явных подсистем



Orchestrator связывает подсистемы; **context manager** решает, что увидит модель; **tool executor** исполняет действия с ограничениями **sandbox**.

ИТЕРАЦИЯ ВМЕСТО «ОДНОГО ОТВЕТА»

Надёжность появляется из замкнутого цикла, а не из единственной генерации



Псевдокод control loop:

```
state := init(request)
while not terminal(state):
  ctx := context_manager(state)
  plan := llm.plan(ctx)
  tool_calls := llm.decide_tools(plan)
  obs := tool_executor.run(tool_calls, policy)
  state := update(state, obs)
  if needs_verify(state):
    state := verification_gate(state)
return artifact(state)
```

Завершение: **Summarize / open PR** — фиксация результата для человека и CI.

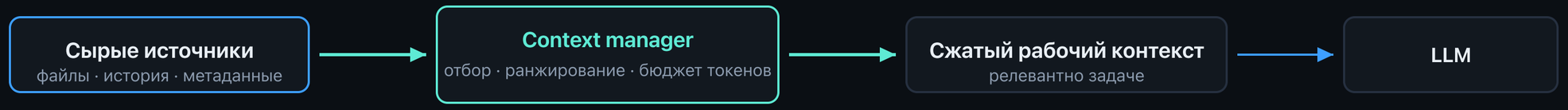
СМЫСЛ VS СЫРОЙ ТЕКСТ

Качество контекста часто важнее размера модели

Навигация по репо
Чтение файлов, карта модулей (repo map), недавние diff.

Поиск смысла
Семантический поиск, индекс символов, граф зависимостей.

Внешний контекст работы
Issue/PR, тикеты, соглашения пользователя и нефункциональные ограничения.



ИНТЕРФЕЙСЫ К МИРУ

Инструменты должны быть явными, логируемыми и жёстко ограниченными

Ввод данных
read file, ripgrep/ast-search, открытие логов.

Изменения
edit / apply_patch, менеджер диффов, узкий scope.

Среда
shell (в песочнице), package manager, браузер/API при политике.

Качество
linter, typecheck, unit/integration/regression tests.

Знания
docs lookup и внутренние справочники, если разрешено сетью.

Governance
каждый вызов: кто, когда, параметры, результат, политика.

Tool executor не «доступ SSH к машине», а контракт: входные схемы, таймауты, allowlist команд, redaction секретов.

ДЕКОМПОЗИЦИЯ

Harness вытягивает из запроса намерение, ограничения и план с учётом рисков

- **Intent:** что меняем и зачем (для пользователя/системы).
- **Constraints:** совместимость API, сроки, флаги, запреты.
- **Affected modules:** слои домена, границы пакетов.
- **Plan:** упорядоченные шаги с критериями готовности.
- **Risk:** миграции данных, breaking changes, флаки-тесты.
- **Rollback:** feature-flag, откат диффа, сохранение поведения.

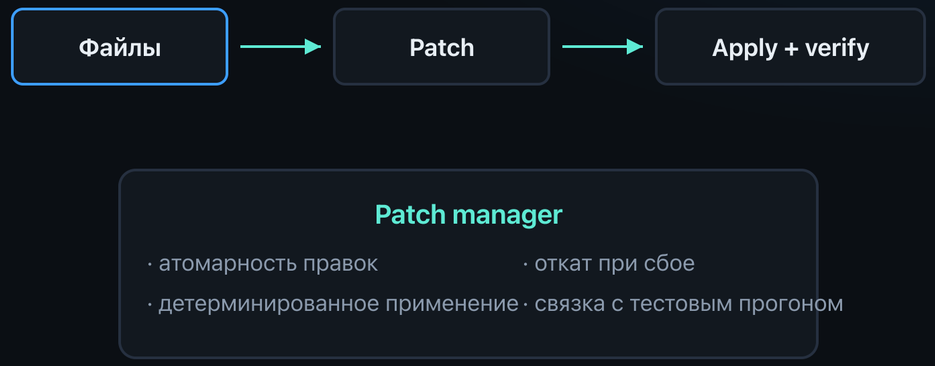
Пример: «Добавить OAuth login»

Схема потока → backend route + хранение сессии → UI → тесты (e2e/unit) → короткая документация для операторов.

МИНИМАЛЬНЫЙ ДИФФ

Агент вносит маленькие проверяемые изменения, а не «переписать всё»

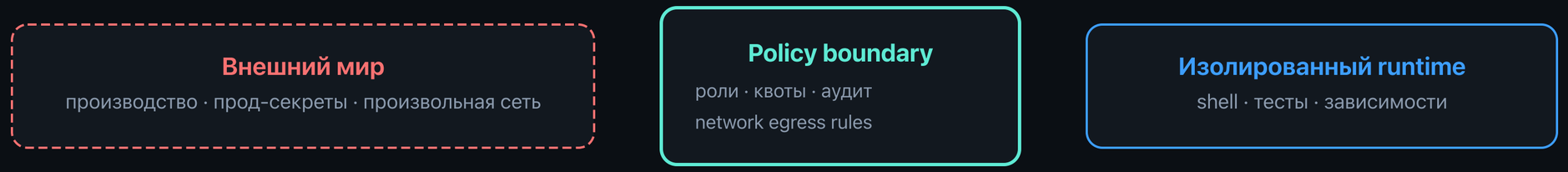
1. Выбрать файлы и границы изменений.
2. Сгенерировать патч (unified diff / structured edit).
3. Применить через **patch manager** с проверкой конфликтов.
4. Перечитать изменённые участки (сверка с намерением).
5. Сузить scope, если дифф разрастается.



ИЗОЛЯЦИЯ

Coding agent должен быть мощным, но не безграничным — граница безопасности в среде исполнения

- **Sandbox:** контейнер/VM/ограниченный пользователь, отдельный workspace, без «лишнего» FS.
- Лимиты CPU/RAM/времени; отмена зависших команд.
- Allowlist/denylist для shell, запрет разрушающих операций без эскалации.
- Секреты: только через vault/переменные среды с маскировкой в логах; сеть — по policy.



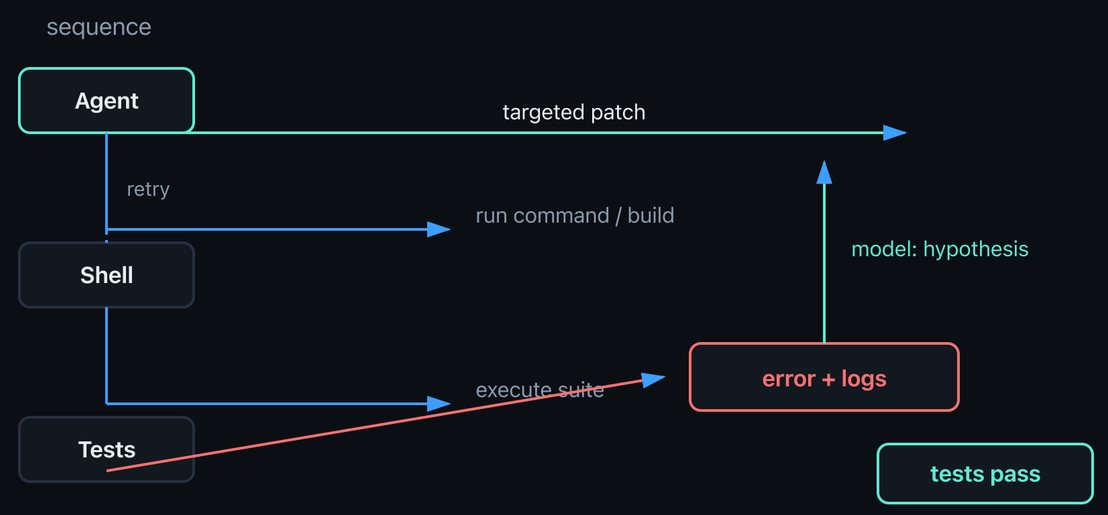
ДОКАЗАТЕЛЬСТВО РАБОТЫ

Harness обязан не только менять код, но и демонстрировать, что изменение выдерживает проверки

Воронка сужает поток: дешёвые проверки ловят очевидное; дорогие тесты подключаются после стабилизации диффа.

ОШИБКА КАК СИГНАЛ

Типичный цикл: падение теста → диагноз → точечный патч → повтор прогона



Псевдокод ремонта:

```
failure := run_tests()
while failure and iterations < budget:
  diag := analyze_logs(failure)
  fix := llm.propose_patch(state, diag)
  if not policy_allows(fix): break
  state := apply_patch(state, fix)
  failure := run_tests()
```

Бюджет итераций и порог «сдаться и эскалировать человеку» — часть политики harness.

GOVERNANCE

Harness — это также граница безопасности между моделью и реальной инфраструктурой

Permission gates
Пошаговые разрешения: чтение → правка → сеть → деплой.

Human-in-the-loop
Утверждение рискованных диффов и команд до исполнения.

Секреты
Нет утечки в промпт/логи; ротация и минимальные scope.

Destructive ops
rm -rf, миграции БД, массовые рефы — отдельный класс риска.

Supply chain
Зависимости и скрипты установки проходят ту же политику, что и агент.

Audit log
Кто, что, когда вызвал; воспроизводимый след для разбора инцидентов.

УПРАВЛЯЕМОЕ УЛУЧШЕНИЕ

Без измерений и evals система не улучшается предсказуемо — только «на ощущениях»

Успех задачи
доля завершённых PR без отката

Тесты
прохождение целевого набора

Итерации
циклов repair до зелёного

Размер патча
строки/файлы относительно базы

Вмешательство человека
escalation rate

Регрессии
дефекты после merge

Time-to-PR
latency пайплайна

Стоимость
токены и CPU на задачу

Evals
Наборы задач с золотым эталоном или критериями «готово»: прогоняются на новых версиях модели и политик, сравнивается регрессия качества.

ПЯТЬ ОПОР

Хороший coding harness держит модель в узде данных, инструментов и обратной связи

Explicit tools	Tight feedback loop	Scoped context	Safe execution	Measurable quality
явные контракты	тесты и логи	релевантность	песочница	метрики и evals

> LLM генерирует гипотезы; harness превращает их в проверяемые изменения кода.